MATERIAL DIDÁCTICO · ACCESO A DATOS

UNIDAD DIDÁCTICA 02

Acceso a bases de datos desde Java

Conectores JDBC, transacciones, mapeo objeto-relacional, bases documentales y componentes de persistencia

GUÍA DE LECTURA

Índice de contenidos

De los repositorios de ficheros de la UD1 a una aplicación que trabaja con bases de datos relacionales y documentales.

01	**De los ficheros a las bases de datos**
02	**Diseño relacional y puesta en marcha del gestor**
03	**JDBC: conexión y operaciones SQL**
04	**Consultas, relaciones y resultados**
05	**Transacciones y procedimientos almacenados**
06	**Separación en capas y patrón DAO**
07	**Persistencia ORM con Hibernate**
08	**Bases de datos objeto-relacionales y de objetos**
09	**Almacenamiento documental con MongoDB**
10	**Componentes reutilizables de acceso a datos**
11	**Proyecto de unidad y comprobaciones**
12	**Secuencia de tutoriales y relaciones**
13	**Migrar información desde la UD1**
14	**Diagnóstico y mantenimiento de la persistencia**

Recorrido de trabajo
Antes de cada relación se realiza el tutorial correspondiente, con un proyecto de ejemplo completo que sirve de referencia. Las relaciones proponen situaciones nuevas que no se resuelven copiando el tutorial.

1. De los ficheros a las bases de datos

En la UD1 construimos aplicaciones capaces de guardar datos en TXT, binario, XML y JSON. También aprendimos a separar el menú de la persistencia mediante un repositorio. Ese contrato será el punto de unión: la aplicación sigue trabajando con objetos **Club**, pero el almacenamiento pasa a una base de datos.

Un fichero funciona muy bien para intercambios sencillos o copias completas. Una base de datos ofrece consultas selectivas, índices, control de acceso, varias sesiones concurrentes, integridad referencial y transacciones. La elección depende del problema: una base de datos no sustituye siempre a un fichero.

Aspecto	Fichero de texto	BD relacional
Buscar un ID	Normalmente recorrer registros	Consulta e índice por clave primaria
Modificar	Puede exigir reescribir	UPDATE de filas seleccionadas
Relaciones	Programadas a mano	Claves externas
Concurrencia	Hay que coordinarla	La gestiona el SGBD
Consistencia	Validación en Java	Restricciones y transacciones

1.1. El desfase objeto-relacional

En Java una entidad se representa con una clase y se accede mediante propiedades. En una base de datos relacional se almacena en **filas** y **columnas**. Esa diferencia se denomina desfase objeto-relacional. Hay dos enfoques que trabajaremos: escribir SQL explícito con JDBC y delegar gran parte de la traducción a Hibernate (ORM).

JAVA / SQL

```
Club club = new Club(1, "Granada CF", "Granada", presupuesto);
// Objeto Java: club.nombre()

SELECT nombre FROM club WHERE id = 1;
-- Tabla SQL: fila con clave primaria 1
```

1.2. Ruta de aprendizaje

El bloque JDBC prepara las bases; después añadiremos transacciones, DAO y ORM. Terminaremos comparando el modelo relacional con los documentos y empaquetando una biblioteca. Los tutoriales usan la misma temática de clubes de la UD1 para centrar el aprendizaje en la tecnología, no en un enunciado nuevo.

2. Diseño relacional y puesta en marcha del gestor

Usaremos un servidor MySQL 8.4 en Docker, un proyecto Maven con Java 17 o superior y un conector JDBC. En los tutoriales encontrarás todos los ficheros, instrucciones de arranque y comprobaciones. Un servidor independiente permite compartir los datos entre varias ejecuciones; una base embebida, como SQLite o H2, almacena los datos sin un servidor separado.

2.1. Nuestro modelo de datos

La tabla **club** tiene una clave primaria **id**; **jugador** guarda una clave externa **club_id**. Un club puede tener varios jugadores, pero cada jugador pertenece a un único club. La tabla **movimiento** registra transferencias de presupuesto entre clubes.

DATOS

```
club(id PK, nombre UNIQUE, ciudad, presupuesto)
jugador(id PK, nombre, dorsal, club_id FK -> club.id)
movimiento(id PK, club_origen_id FK, club_destino_id FK,
           importe, fecha)
```

La clave primaria identifica de forma única un registro. La clave externa exige que el club asignado a un jugador exista. La restricción **UNIQUE** impide dos nombres iguales y **CHECK** impide presupuestos negativos. No basta con validar estos casos en el menú: también interesa que la base de datos los garantice.

2.2. ¿Qué crean los scripts del tutorial?

El archivo **compose.yaml** descarga e inicia MySQL, crea un volumen para persistir datos y publica el puerto 3307 solo en el ordenador local. **01_esquema.sql** crea las tablas, inserta tres clubes y tres jugadores y define el procedimiento **contar_jugadores**. El SQL inicial se ejecuta automáticamente solo la primera vez que se crea el volumen.

TERMINAL

```
cd Codigo/Infraestructura/mysql
docker compose up -d
docker compose ps
# Consulta interactiva al servidor:
docker compose exec mysql mysql -uad_user -p ad_ud2
```

> **Punto de atención**
> Si modificas 01_esquema.sql cuando el volumen ya existe, no se ejecutará de nuevo. No borres el volumen por rutina: docker compose down -v destruye los datos de prácticas. Para crear tablas adicionales, aplica un script SQL nuevo de forma consciente.

2.3. Tablas normalizadas frente a documentos

En el modelo relacional evitamos repetir la ciudad y el nombre del club en cada jugador: el campo club_id permite reconstruir la relación mediante un JOIN. En MongoDB veremos una alternativa documental en la que algunos datos se incrustan dentro de un documento para acelerar la recuperación conjunta. No existe un diseño universalmente mejor.

3. JDBC: conexión y operaciones SQL

JDBC (Java Database Connectivity) es la API estándar de Java para comunicarse con motores SQL mediante controladores. Nuestra aplicación habla con las interfaces java.sql; el conector MySQL traduce las llamadas al protocolo del servidor. La dependencia se incorpora con Maven y queda fijada en el archivo pom.xml.

Objeto JDBC	Función principal
DriverManager	Solicitar una conexión a través del driver disponible
Connection	Representar una sesión y controlar transacciones
PreparedStatement	Enviar SQL parametrizado
ResultSet	Recorrer filas devueltas por SELECT
CallableStatement	Ejecutar procedimientos almacenados

3.1. Establecer conexión

JAVA

```
try (Connection conexion = DriverManager.getConnection(
        "jdbc:mysql://localhost:3307/ad_ud2?connectionTimeZone=UTC",
        "ad_user", "ad_pass")) {
    System.out.println(conexion.isValid(2));
} catch (SQLException e) {
    System.err.println(e.getMessage());
}
```

La cadena JDBC contiene el protocolo, host, puerto y esquema. El usuario y su contraseña corresponden al servidor local que levanta Docker. **try-with-resources** cierra el recurso incluso si aparece una excepción. Abrir y cerrar conexiones tiene un coste; en aplicaciones mayores se usa **pooling**, una reserva de conexiones administradas por una biblioteca.

3.2. Por qué PreparedStatement en lugar de concatenar

Nunca construiremos SQL uniendo directamente cadenas de entrada del usuario. **PreparedStatement** separa el texto de la sentencia de los valores, ayuda a evitar inyección SQL y aporta métodos de conversión de tipos como setInt, setString y setBigDecimal.

JAVA

```
String sql = "SELECT id, nombre, ciudad, presupuesto " +
             "FROM club WHERE id = ?";
try (PreparedStatement st = conexion.prepareStatement(sql)) {
    st.setInt(1, 2);
    try (ResultSet filas = st.executeQuery()) {
        if (filas.next()) {
            String nombre = filas.getString("nombre");
        }
    }
}
```

El signo **?** es un marcador de posición. El índice de parámetros empieza en **1**, no en cero. El método executeQuery se usa cuando esperamos un conjunto de filas. Para INSERT, UPDATE y DELETE utilizaremos executeUpdate, que devuelve el número de filas afectadas.

3.3. Altas, consultas, cambios y borrados

SQL

```
INSERT INTO club(nombre, ciudad, presupuesto) VALUES (?, ?, ?);
SELECT id, nombre, ciudad, presupuesto FROM club ORDER BY id;
UPDATE club SET ciudad = ? WHERE id = ?;
DELETE FROM club WHERE id = ?;
```

Al insertar con una clave autonumérica, podemos solicitar **Statement.RETURN_GENERATED_KEYS** y recuperar el nuevo ID con **getGeneratedKeys()**. Al modificar o borrar, comprobar el número de filas afectadas permite distinguir entre una operación aplicada y un ID que no existe.

> **Decisión de diseño**
> El código de acceso a datos devuelve Club, List<Club>, Optional<Club> o boolean. No imprime resultados: mostrar mensajes es responsabilidad de Main o de la interfaz del programa.

4. Consultas, relaciones y resultados

JDBC devuelve las filas de una consulta mediante **ResultSet**. Se empieza antes de la primera fila y se avanza con next(). Los métodos getInt, getString y getBigDecimal recuperan valores según su tipo. Para no devolver un ResultSet asociado a una conexión que pronto se cerrará, lo convertimos a objetos Java dentro del método DAO.

JAVA

```
List<Club> clubes = new ArrayList<>();
while (filas.next()) {
    clubes.add(new Club(
        filas.getInt("id"),
        filas.getString("nombre"),
        filas.getString("ciudad"),
        filas.getBigDecimal("presupuesto")));
}
return clubes;
```

4.1. Consultas con JOIN

Los datos de un jugador y de su club están en tablas distintas. JOIN utiliza la clave externa para combinar filas relacionadas. El alias facilita leer la consulta, especialmente si diferentes tablas comparten el nombre de alguna columna.

SQL

```
SELECT J.nombre AS jugador, J.dorsal, C.nombre AS club
FROM jugador J
JOIN club C ON J.club_id = C.id
WHERE C.ciudad = ?
ORDER BY J.nombre;
```

4.2. Agregaciones y resultados especiales

SQL

```
SELECT C.nombre, COUNT(J.id) AS total
FROM club C
LEFT JOIN jugador J ON C.id = J.club_id
GROUP BY C.id, C.nombre
ORDER BY total DESC;
```

LEFT JOIN conserva los clubes sin jugadores; COUNT(J.id) devuelve cero en esos casos. En Java se puede crear un record **ResumenClub(String nombre, long total)** para almacenar cada fila. Un resultado que no existe se representa mediante Optional.empty(), una lista vacía o una excepción si es un error de negocio: no son equivalentes y conviene elegirlo deliberadamente.

4.3. Excepciones y pruebas

SQLException puede deberse a que el servidor no está iniciado, a una contraseña incorrecta, a una violación de UNIQUE o a un fallo de comunicación. No conviene capturar cualquier excepción y continuar como si nada: el DAO debe informar del problema y el programa decide cómo presentarlo.

- Prueba una búsqueda inexistente y comprueba que no hay IndexOutOfBoundsException ni NullPointerException.
- Prueba una inserción con un nombre repetido: MySQL debe rechazarla por la restricción UNIQUE.
- Cierra MySQL y comprueba que la aplicación comunica el error y no simula que el dato se guardó.

El **Tutorial 01** resuelve el proyecto completo y prepara la **Relación 01**.

5. Transacciones y procedimientos almacenados

Una transacción reúne varias operaciones de modo que se aplican todas o no se aplica ninguna. En nuestro ejemplo, transferir 25 euros de un club a otro exige descontar, abonar y registrar el movimiento. No sería aceptable descontar el dinero y perder el resto del proceso por un error posterior.

5.1. Cuatro propiedades ACID

Propiedad	Significado
Atomicidad	Todas las operaciones de la transacción o ninguna
Consistencia	Las restricciones válidas se siguen cumpliendo
Aislamiento	La concurrencia se controla según el nivel configurado
Durabilidad	Tras COMMIT, los cambios confirmados persisten

5.2. Commit y rollback

JAVA

```
try (Connection con = Conexion.abrir()) {
    con.setAutoCommit(false);
    try {
        // Descontar origen, abonar destino, registrar movimiento.
        con.commit();
    } catch (SQLException e) {
        con.rollback();
        throw e;
    }
}
```

El ejemplo anterior muestra la estructura, no la operación completa. En el proyecto resuelto verás los tres PreparedStatement. Usamos la misma **Connection** en toda la transacción, porque cada conexión mantiene su propia transacción. Con autocommit desactivado, cada sentencia no se confirma por separado.

Para que una transferencia falle sin corromper datos, el descuento incorpora la condición **presupuesto >= importe**. Comprobamos además que el UPDATE de origen y el de destino afecten exactamente a una fila. Si el club destino no existe, la operación debe recuperar el presupuesto original con rollback.

5.3. Procedimientos almacenados y parámetros

Un procedimiento almacenado es código SQL que reside en el servidor y puede admitir parámetros **IN**, **OUT** o **INOUT**. En el SQL de arranque se define contar_jugadores(IN p_club_id, OUT p_total); JDBC lo ejecuta mediante CallableStatement.

JAVA

```
try (CallableStatement st = con.prepareCall(
        "{call contar_jugadores(?, ?)}")) {
    st.setInt(1, clubId);
    st.registerOutParameter(2, Types.INTEGER);
    st.execute();
    return st.getInt(2);
}
```

No se debe confundir un procedimiento del servidor con un método Java. Cambiar el procedimiento requiere acceso y permisos sobre MySQL, mientras que cambiar un método DAO implica desplegar una nueva versión del programa.

5.4. Gestión de concurrencia

La comprobación y el descuento se realizan en un único UPDATE condicionado. Esto evita el patrón frágil de consultar el saldo en una conexión y descontarlo mucho después en otra. En operaciones más complejas pueden ser necesarios bloqueos de filas (SELECT ... FOR UPDATE), órdenes consistentes de bloqueo, reintentos controlados o niveles de aislamiento específicos.

El **Tutorial 02** muestra una transferencia real y una fallida, comprueba el rollback y llama al procedimiento. Después se realiza la **Relación 02**.

6. Separación en capas y patrón DAO

Una aplicación de gestión combina varias responsabilidades: recoger datos, aplicar reglas de negocio y acceder al almacenamiento. Si Main mezcla Scanner, SQL, excepciones y salida de texto, las pruebas y el mantenimiento se complican. El patrón **DAO** (Data Access Object) concentra el acceso a datos.

DIAGRAMA

```
Main / UI
    | llama
ServicioClubes (reglas de negocio y transacciones)
    | utiliza
RepositorioClubes (interfaz)
    | implementado por
RepositorioMysql (JDBC)  /  RepositorioOrm (Hibernate)
```

La UD1 introdujo la interfaz RepositorioClubes con guardarTodos() y cargarTodos(). Ahora podemos definir métodos específicos como buscar(int id) o listar(), pues el servidor resuelve consultas selectivas sin cargar todo el contenido en memoria.

6.1. Contrato e implementación

JAVA

```
public interface RepositorioClubes {
    List<Club> listar() throws SQLException;
    Optional<Club> buscar(int id) throws SQLException;
}
```

La interfaz es una promesa sobre operaciones disponibles, no sobre cómo se guardan los datos. La implementación puede utilizar JDBC, ORM, ficheros o incluso una API remota. El resto del programa depende del contrato, no del SQL específico.

6.2. Gestión correcta de recursos

Los objetos Connection, Statement y ResultSet ocupan recursos del lado del programa y del servidor. Se cierran con try-with-resources, sin devolverlos al menú. Para consultas pequeñas de aula basta con abrir una conexión por método; en servicios concurrentes usaríamos un pool y externalizaríamos credenciales, límites y tiempos de espera.

6.3. Tipos numéricos, fechas y validaciones

Para cantidades monetarias se recomienda **BigDecimal**, no float ni double: los tipos binarios pueden producir pequeñas imprecisiones. En la base de datos utilizamos DECIMAL(10,2). Para fechas nuevas en Java conviene la API java.time, por ejemplo LocalDate, LocalDateTime e Instant; sus conversiones con SQL deben atender al tipo de columna y a la zona horaria.

> **Regla de clase**
> Los métodos de acceso a datos no contienen System.out.println. Devuelven objetos, listas, resultados o excepciones; Main es el lugar donde se muestran mensajes.

En el **Tutorial 05** convertiremos este diseño en una biblioteca Maven real que se instala y se utiliza desde otro proyecto.

7. Persistencia ORM con Hibernate

Un **ORM** (Object-Relational Mapping) relaciona clases y asociaciones Java con tablas y claves externas. Hibernate es una implementación conocida. Permite crear, buscar, actualizar y borrar entidades usando sesiones y operaciones sobre objetos; también ofrece JPQL/HQL para consultas. No elimina SQL ni las decisiones sobre índices y rendimiento.

7.1. Entidades y anotaciones

Una entidad es una clase cuyos objetos tienen identidad persistente. Debe declarar un identificador y disponer de un constructor sin argumentos accesible para Hibernate. Las anotaciones principales son @Entity, @Table, @Id, @GeneratedValue, @Column, @ManyToOne y @JoinColumn.

JAVA

```
@Entity
@Table(name = "club")
public class Club {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer id;

    @Column(nullable = false, unique = true)
    private String nombre;

    protected Club() { }
    // Constructor de negocio, getters y otros campos.
}
```

La anotación **@ManyToOne** en Jugador indica que varios jugadores pueden referirse al mismo Club. **@JoinColumn(name="club_id")** designa la columna física. Usamos FetchType.LAZY para no cargar automáticamente todo el objeto Club cada vez que se consulta un jugador.

7.2. Sesión, estado y ciclo de vida

Estado	Interpretación
Transient	Objeto creado en Java, aún sin persistir
Managed	Asociado a una sesión; Hibernate controla cambios
Detached	Objeto existente, pero fuera de la sesión
Removed	Marcado para borrado al confirmar

Una **SessionFactory** se construye una vez por configuración y tiene un coste elevado. A partir de ella se abren sesiones cortas; una transacción comienza con beginTransaction(), se confirma con commit() y se revierte con rollback().

JAVA

```
try (Session sesion = factoria.openSession()) {
    Transaction tx = sesion.beginTransaction();
    try {
        Club club = sesion.find(Club.class, 1);
        club.setCiudad("Granada");
        tx.commit();  // Hibernate detecta cambios
    } catch (RuntimeException e) {
        if (tx.isActive()) tx.rollback();
```

```
            throw e;
        }
    }
```

En un objeto **managed**, Hibernate puede detectar cambios y emitir UPDATE al sincronizar la sesión. Es lo que se conoce como **dirty checking**. Si el objeto está detached, modificar sus campos no garantiza que el cambio se guarde sin volver a vincularlo al contexto de persistencia.

7.3. Consultas JPQL/HQL

JPQL

```
select j.nombre
from Jugador j join j.club c
where c.ciudad = :ciudad
order by j.nombre
```

JPQL consulta **entidades y propiedades**, no nombres de tablas directamente. :ciudad es un parámetro enlazado con setParameter. Para conocer el SQL generado podemos activar show_sql durante el aprendizaje, pero en producción no conviene depender de ese registro ni imprimir datos sensibles.

7.4. Estrategia de creación del esquema

Para esta UD2 la base se crea mediante **01_esquema.sql** y Hibernate se configura con **hibernate.hbm2ddl.auto=validate**. Así comprueba el esquema, pero **no lo modifica**. No usaremos create-drop como opción por defecto: podría borrar datos de otros ejercicios.

7.5. Limitaciones del ORM

Un ORM reduce código repetitivo, pero puede esconder consultas innecesarias (problema N+1), crear dependencias de versión y requerir atención al ciclo de vida y a las transacciones. Para consultas intensivas o SQL muy específico puede seguir conveniendo JDBC. En las relaciones se valorará también la consulta generada y el cierre correcto de sesiones.

El **Tutorial 03** configura Hibernate desde cero, reutiliza las tablas de MySQL, mapea Club/Jugador y ejecuta consultas y cambios completos.

8. Bases de datos objeto-relacionales y de objetos

No debemos confundir tres modelos distintos: **ORM** es una técnica para llevar objetos Java a una base relacional; una base **objeto-relacional** es un gestor relacional con tipos y funciones más ricos; y una base **orientada a objetos** almacena estructuras y referencias de objetos con sus propios mecanismos de consulta y persistencia.

Tecnología	Idea principal	Ejemplo
SQL relacional	Tablas, columnas, PK y FK	MySQL
Objeto-relacional	SQL más tipos compuestos, arrays o JSON	PostgreSQL
Orientada a objetos	Persistencia nativa de objetos y referencias	Gestores OODB
ORM	Mapea objetos sobre tablas SQL	Hibernate/JPA

8.1. Ejemplo objeto-relacional: JSONB en PostgreSQL

PostgreSQL es un gestor relacional que permite columnas JSONB e indexación sobre sus valores. Esta posibilidad permite combinar estructura relacional con atributos semi-estructurados cuando está justificado. El ejemplo siguiente es ilustrativo, no forma parte del entorno MySQL de los tutoriales.

SQL / POSTGRESQL

```
CREATE TABLE club_ext (
    id BIGSERIAL PRIMARY KEY,
    nombre TEXT NOT NULL,
    datos JSONB NOT NULL
);
INSERT INTO club_ext(nombre, datos)
VALUES ('Granada CF', '{"colores":["rojo","blanco"]}');
SELECT nombre, datos->>'colores' AS colores FROM club_ext;
```

8.2. Bases orientadas a objetos y OQL

En un modelo orientado a objetos se persisten clases, objetos y relaciones sin exigir la representación tradicional en tablas. OQL es un lenguaje de consultas asociado históricamente a este modelo; su soporte y sintaxis real dependen del gestor. No lo trataremos como una API universal que funcione en MySQL, Hibernate o MongoDB.

8.3. Alcance de esta unidad

El resultado de aprendizaje referido a bases objeto-relacionales y orientadas a objetos es parte del currículo. Con la extensión solicitada, aquí se abordan **las diferencias conceptuales** y un ejemplo de datos objeto-relacionales. Para evaluar de manera íntegra el desarrollo con un SGBD nativo de objetos habría que incorporar un laboratorio adicional sobre un producto concreto. No se sustituye ese criterio por MongoDB: una base documental no es lo mismo que una base orientada a objetos.

9. Almacenamiento documental con MongoDB

Una base documental guarda información en **documentos** y los agrupa en **colecciones**. MongoDB representa los documentos mediante BSON, una codificación binaria con tipos adicionales a JSON. Un documento puede tener listas y subdocumentos; no necesita que todos los documentos de una colección contengan exactamente los mismos campos.

BSON / JSON

```
{
  "_id": 1,
  "nombre": "Granada CF",
  "ciudad": "Granada",
  "jugadores": [
    { "nombre": "Lucía", "dorsal": 10 },
    { "nombre": "Pablo", "dorsal": 7 }
  ]
}
```

9.1. ¿Incrustar o referenciar?

En MySQL, jugador.club_id establece una relación mediante clave externa. En el ejemplo MongoDB los jugadores se incrustan en el mismo documento del club. Incrustar favorece leer todo el conjunto a la vez y, en ciertos casos, actualizarlo atómicamente dentro de un documento. Referenciar favorece gestionar elementos compartidos o que crecen sin control. El diseño se decide por las consultas y el patrón de escritura.

9.2. Conexión y colecciones

El tutorial usa el driver oficial síncrono de MongoDB para Java y un servidor local en Docker publicado solo en 127.0.0.1:27018. La colección se obtiene con MongoDatabase.getCollection("club"); las operaciones CRUD devuelven documentos, resultados de actualización o cantidades de registros afectados.

JAVA

```
try (MongoClient cliente =
        MongoClients.create("mongodb://localhost:27018")) {
    MongoCollection<Document> clubes = cliente
        .getDatabase("ad_ud2").getCollection("club");
```

```
    Document club = clubes.find(eq("_id", 1)).first();
}
```

9.3. CRUD documental y filtros

JAVA

```
coleccion.insertOne(documento);
coleccion.find(eq("ciudad", "Granada"));
coleccion.updateOne(eq("_id", 1), set("ciudad", "Granada"));
coleccion.deleteOne(eq("_id", 3));
```

Para iniciar datos reproducibles, el tutorial utiliza **replaceOne** con **upsert(true)**, que inserta si no existe el documento y sustituye su versión anterior si ya existe. Para una actualización parcial se utiliza updateOne; no confundas reemplazar un documento completo con cambiar un campo concreto.

9.4. Integridad, transacciones y seguridad

La ausencia de clave externa SQL no significa que una aplicación documental no tenga reglas de integridad. Debemos validar campos, definir índices únicos cuando sea necesario y elegir con cuidado las operaciones compuestas. MongoDB admite transacciones multidocumento en despliegues compatibles; el ejemplo local utiliza solo operaciones sobre documentos individuales.

La instancia de aula se ejecuta sin autenticación y se publica solo en loopback. No se debe hacer accesible desde Internet ni usar credenciales demostrativas en producción. En el **Tutorial 04** prepararás el proyecto y ejecutarás el CRUD completamente resuelto.

9.5. Comparación final

Necesidad	Relacional	Documental
Integridad entre entidades	FK y restricciones	Reglas e índices, a menudo en la aplicación
Consultar relaciones	JOIN	Incrustación, referencias, aggregation
Estructura	Esquema explícito	Documentos flexibles
Mejor cuando	Consultas relacionales y transacciones	Agregados documentales y estructura cambiante

10. Componentes reutilizables de acceso a datos

Un **componente** es una pieza de software con una interfaz definida que se puede incorporar a más de una aplicación. Separar el acceso a datos en una biblioteca evita duplicar SQL en varios proyectos y permite sustituir la implementación sin cambiar la forma en que se consulta la información.

10.1. Empaquetar una biblioteca Maven

Un proyecto con empaquetado JAR (valor predeterminado de Maven) compila clases bajo src/main/java y genera un archivo JAR en target. El comando **mvn clean install** además publica la biblioteca en el repositorio Maven local del ordenador. Un segundo proyecto la añade como dependencia mediante groupId, artifactId y version.

XML / MAVEN

```
<dependency>
  <groupId>es.curso.ad</groupId>
  <artifactId>componente-clubes</artifactId>
  <version>1.0-SNAPSHOT</version>
</dependency>
```

10.2. Características de un buen componente

- Interfaz pública pequeña, clara y documentada; una implementación puede cambiar sin romper consumidores.
- Ausencia de impresión o interacción con el usuario dentro de la biblioteca.
- Gestión explícita de conexiones, errores y parámetros configurables.
- Pruebas que comprueben resultados y errores, no solo mensajes en pantalla.
- Versión, nombre de paquete y dependencia Maven identificables.

10.3. Composición y dependencias

En el proyecto resuelto el consumidor declara una variable de tipo **RepositorioClubes** y recibe una instancia de **RepositorioMysql**. Este es un ejemplo sencillo de programación contra interfaces. El consumidor solo conoce listar() y buscar(), no sus sentencias SQL.

JAVA

```
RepositorioClubes repositorio =
    new RepositorioMysql(url, usuario, clave);
List<Club> clubes = repositorio.listar();
```

El **Tutorial 05** contiene dos proyectos Maven completos, instrucciones de instalación local y ejecución del consumidor. Sirve de preparación para la **Relación 05**.

11. Proyecto de unidad: gestor de clubes con persistencia intercambiable

Amplía el gestor de clubes de la UD1 para que trabaje principalmente con MySQL. Debe mantener la separación del menú y la persistencia; se añadirá un módulo de jugadores y una operación transaccional sobre presupuestos. La versión documental y ORM permiten comparar distintas implementaciones sin rehacer toda la lógica de negocio.

11.1. Opciones de funcionamiento

DATOS

```
GESTOR DE CLUBES · UD2
1. Crear club
2. Listar clubes y jugadores
3. Buscar club por ID
4. Cambiar ciudad
5. Borrar club (si las restricciones lo permiten)
6. Fichar jugador
7. Transferir presupuesto entre clubes
8. Ver resumen agrupado por club
9. Exportar listado a JSON (repaso de UD1)
0. Salir
```

11.2. Condiciones mínimas

- La persistencia resiste cerrar y volver a ejecutar Java; ningún CRUD depende de listas en memoria como fuente definitiva.
- No se concatena ningún dato del usuario al texto SQL; se usan parámetros preparados.
- Los IDs inexistentes, nombres repetidos y errores del servidor generan respuestas diferenciadas.
- La transferencia hace commit si todas las escrituras funcionan y rollback si alguna falla.
- El código del menú no contiene SQL, DriverManager, Session ni MongoCollection.
- Los recursos se cierran correctamente y el resultado de las consultas se transforma a objetos Java.

11.3. Plan de pruebas

Caso	Resultado esperado
Crear y recuperar	El club persiste después de reiniciar
Nombre duplicado	Error de integridad; sin segunda fila
Transferencia a ID inexistente	Rollback; presupuestos sin cambios
Borrar club con jugadores	Fallo controlado por FK RESTRICT
SQL malicioso en nombre	Se trata como texto, no como sentencia
Desconexión del servidor	Error comunicado; sin falso éxito

11.4. Evidencias de entrega

Entrega el repositorio con README de instalación, scripts SQL, código Java organizado por capas, capturas o transcripciones de pruebas y un breve documento donde justifiques cuándo utilizar JDBC, Hibernate o MongoDB. No se entregan credenciales reales ni carpetas target/.m2.

12. Secuencia de tutoriales y relaciones

El material complementario está organizado para que primero se realice un tutorial completamente resuelto y después una relación de trabajo autónomo. Los proyectos del directorio Codigo contienen las soluciones íntegras de los tutoriales; no son soluciones de las relaciones.

Orden	Tutorial previo	Relación posterior
1	T01: MySQL + JDBC + CRUD	R01: consultas y CRUD
2	T02: transacciones y procedimientos	R02: operaciones atómicas
3	T03: Hibernate ORM	R03: entidades y JPQL
4	T04: MongoDB documental	R04: colecciones y documentos
5	T05: componente Maven y consumo	R05: biblioteca y pruebas

12.1. Alineación con el temario oficial

RA	Tratamiento en la UD2
RA2 · Conectores relacionales	JDBC, consultas, modificaciones, procedimientos y transacciones
RA3 · ORM	Hibernate, entidades, sesiones, JPQL y transacciones
RA4 · Objeto-relacional / OO	Panorama y ejemplo PostgreSQL; no laboratorio OODB completo
RA5 · Bases documentales	MongoDB: colecciones, documentos, CRUD y filtros
RA6 · Componentes de acceso	Contrato, JAR, publicación Maven y uso desde otra aplicación

Normativa utilizada como contraste: **Real Decreto 405/2023, de 29 de mayo** (actualización del módulo profesional 0486, Acceso a datos). La distribución aquí presentada es una **organización didáctica en dos unidades**, no una transcripción oficial de unidades. La UD1 abarca principalmente RA1.

12.2. Reglas de trabajo

- Ejecuta primero los tutoriales sin modificar el código; localiza los datos reales y comprueba la salida.
- Repite la ejecución e identifica qué operaciones son idempotentes y cuáles alteran el estado cada vez.
- En las relaciones escribe los métodos de persistencia como funciones que devuelven resultados; muestra la información solo en Main.
- Prueba los casos incorrectos antes de considerar terminado el ejercicio.

Al terminar
Serás capaz de conectar Java a una base de datos, realizar CRUD seguro, gestionar transacciones, comparar JDBC y ORM, trabajar con documentos y empaquetar una capa de persistencia reutilizable.

13. Migrar información desde la UD1

Una base de datos no siempre nace vacía. A veces tenemos que recuperar información de una aplicación anterior, como el fichero de clubes de la UD1, y llevarla al nuevo sistema sin perder registros. Esta operación es una **migración** y necesita un tratamiento más exigente que copiar líneas de texto.

13.1. Diseñar el contrato de importación

El formato de origen es **id;nombre;ciudad**, codificado en UTF-8. El destino es la tabla SQL club; el presupuesto, que no existía en el TXT original, se inicializa a cero. El primer paso consiste en leer y validar todas las líneas, sin tocar todavía MySQL. El segundo consiste en insertar todas las filas dentro de una transacción.

Fase	Comprobación
Lectura	Existe el fichero, UTF-8 y acceso permitido
Validación	Tres campos; ID entero positivo; nombre y ciudad no vacíos
Consistencia en lote	No hay IDs repetidos dentro del TXT
Persistencia	INSERT parametrizado con una única Connection
Error	Rollback íntegro; ningún registro parcial
Confirmación	Commit solo cuando se han insertado todos

13.2. Ejemplo de entrada y salida

TXT

```
101;Córdoba CF;Córdoba
102;Real Jaén;Jaén
103;Recreativo de Huelva;Huelva
```

Estos registros se incluyen en **Codigo/T01_JDBC_CRUD/datos/clubes_importacion.txt**. La clase **ImportarTxt** tiene dos métodos: validar(Path), que devuelve List<ClubTxt>, e importar(Path), que devuelve el número de elementos insertados o lanza una excepción. No imprime desde esos métodos.

JAVA · ESQUEMA

```
List<ClubTxt> datos = validar(archivo);
try (Connection conexion = Conexion.abrir()) {
    conexion.setAutoCommit(false);
    try (PreparedStatement st = conexion.prepareStatement(
            "INSERT INTO club(id,nombre,ciudad,presupuesto) " +
            "VALUES (?, ?, ?, ?)")) {
        for (ClubTxt club : datos) {
            // Parámetros: ID, nombre, ciudad y BigDecimal.ZERO.
            // Un executeUpdate() por club.
        }
        conexion.commit();
    } catch (SQLException e) {
        conexion.rollback();
        throw e;
    }
}
```

La implementación íntegra está en la carpeta Codigo. Para ejecutarla sin tocar Main, utiliza **mvn compile exec:java -Dexec.mainClass=es.curso.ad.t01.MainImportacion** desde T01_JDBC_CRUD. Después consulta la tabla club en MySQL y verifica que existen los IDs 101, 102 y 103.

13.3. Comportamiento ante errores

Si vuelves a ejecutar la importación, MySQL rechazará los IDs ya existentes. Eso es correcto: no debe crear duplicados ni cambiar datos sin una política explícita. Como las inserciones se realizan en una sola transacción, el lote queda sin escrituras parciales. Para repetir desde cero, elimina conscientemente esos tres registros de prueba, no todo el volumen de Docker.

Prueba también un TXT con un ID duplicado dentro del propio fichero o una línea con solo dos campos. La validación debe fallar **antes de abrir la conexión**. Esta secuencia recupera una idea de la UD1: nunca sustituir información válida por datos importados que no han sido comprobados.

14. Diagnóstico y mantenimiento de la persistencia

Cuando un programa no funciona, el objetivo no es modificar código al azar. Hay que aislar qué capa está fallando: proceso Java, dependencia Maven, conector, conexión de red, esquema SQL, restricciones o consulta. La siguiente tabla ofrece una ruta inicial de comprobación.

Síntoma	Primera comprobación
Communications link failure	docker compose ps; puerto 3307 y estado del servidor
Access denied for user	Usuario y clave del contenedor; permisos SQL
Unknown database	Nombre del esquema ad_ud2 y ejecución inicial del SQL
Duplicate entry	Restricciones UNIQUE y valores que ya existen
Cannot add/update a child row	FK de jugador.club_id y existencia del club
Hibernate schema validation error	Tablas y columnas físicas; no activar create-drop a ciegas
MongoDB timeout	Servidor Mongo activo y puerto 27018 correcto

Síntoma	Primera comprobación
Maven cannot resolve dependency	Conexión a Maven Central, pom.xml y versión del artefacto

14.1. Comprobaciones reproducibles

TERMINAL / SQL

```
docker compose ps
docker compose logs --tail=80 mysql

# En el proyecto Maven
mvn -version
mvn clean compile

# Dentro de MySQL
SHOW TABLES;
SELECT COUNT(*) FROM club;
SHOW CREATE TABLE jugador;
```

Hay que ejecutar cada comando desde la carpeta correcta: docker compose actúa sobre el compose.yaml del directorio de trabajo; Maven sobre el pom.xml. Si las rutas relativas del programa no son las esperadas, consulta el directorio de ejecución del IDE, no modifiques todas las rutas a absolutos particulares.

14.2. Rendimiento e índices

La clave primaria del club crea un índice que favorece las búsquedas por ID. Una consulta por ciudad podría necesitar un índice adicional si la tabla crece y se consulta con frecuencia. Para observar el plan de una consulta se utiliza EXPLAIN; crear índices sin atender a las consultas reales también tiene costes de espacio y mantenimiento.

SQL

```
EXPLAIN SELECT * FROM club WHERE ciudad = 'Granada';
CREATE INDEX idx_club_ciudad ON club(ciudad);
EXPLAIN SELECT * FROM club WHERE ciudad = 'Granada';
```

14.3. Seguridad y evolución

Las contraseñas demostrativas sirven para un entorno privado de aula, nunca para una aplicación desplegada. En producción deben mantenerse fuera del código fuente, utilizar usuarios con permisos mínimos, cifrar las comunicaciones cuando corresponda y ejecutar migraciones de esquema controladas. Guardar o hacer copias de la base de datos es diferente de copiar el JAR de Java.

14.4. Comprobación final del aprendizaje

- Sé explicar por qué en la UD1 reescribíamos líneas y ahora usamos sentencias SQL sobre filas concretas.

- Identifico el driver, Connection, PreparedStatement, ResultSet y CallableStatement en un proyecto real.
- Soy capaz de provocar un rollback y demostrar que la base de datos no queda a medias.
- Entiendo qué traduce Hibernate, cuándo una entidad está managed y qué devuelve una consulta JPQL.
- Distingo entre tabla, entidad ORM, objeto persistente y documento BSON.
- Soy capaz de publicar un repositorio de acceso a datos como un JAR e integrarlo en otra aplicación.

Cierre de unidad
La persistencia no consiste únicamente en guardar datos: exige decidir cómo se modelan, recuperan, protegen y mantienen sin romper las reglas de negocio.